Implementation of Bit Depth and Statistical Hiding Steganography for Bitmap Images

Group 6

Project Final Report

CS-4463-01T-Summer-2021-Steganography

Group Members:
Ben Baggs
Rongrong Wei

Date: August 6, 2021

Introduction

For our project the objective was to explore various strategies for embedding a message using a full color bitmap file as a cover using Python. Our group was under the impression we would be on our own and without source code or help while we more or less pioneered a strategy for embedding using Python. Our group was able to accomplish multiple methods of encoding bitmaps by using a serialization library built by Rongrong, and using pythons PIL or Python Image Library. We accepted our professor's advice and set both of his suggestions as our goals for the final project. First, we are able to embed multiple messages into multiple bitlayers within the bitmap and second we implemented statistical hiding methods, both average and sum, to embed our message. Ultimately, we were able to accomplish both techniques and hopefully pioneer a codebase that future classes will be able to use as reference should they choose to utilize Python for bitmap steganography.

Code Base Description

Setup

To set up the program, the program needs to use Python 3 (not Python 2) and the Python image library Pillow. To install Pillow, the following command can be used:

python -m pip install Pillow

or, if there are multiple versions of python:

python3 -m pip install Pillow

Using the Program

In this project, we created two python programs. One program “encode.py” can be used to hide a message using a bit-depth method. The example syntax for running this code is as follows.

To encode:

```
python encoder.py encode colortest.bmp cat.jpg --bit_depth 7
```

To decode, the output file (`Encoded_output.bmp`):

```
python encoder.py decode Encoded_output.bmp out --bit_depth 7
```

The decoder will save the hidden message in a file named `output.xxx`. Where xxx is the original message’s file extension. The encode.py can hide at any bit depth value from 1 to 8.

The other program, “average.py” uses statistical hiding to hide a message in a cover file. The example syntax for running this code is as follows.

To encode:

```
python average.py encode colortest.png --message cat.jpg --lsb 1
--pixelno 9 --method avg
```

To decode, the output cover file (`stego.png`):

```
average.py decode stego.png --lsb 1 --pixelno 9 --method avg
```

The decoder will save the hidden message in a file named `message_extract.xxx`. Where xxx is the original message's file extension. The average.py program can hide with different settings. The option `--lsb` can be set to any number between 1 and 7 and the option `--pixelno` can be set to any value between 1 and n where n is the total number of pixel components (rgb) in the image. The `--method` setting can be set to `avg` to use average statistical hiding and to `sum` use the sum statistical hiding.

Functional Block Diagrams

Fig 1. Encoding

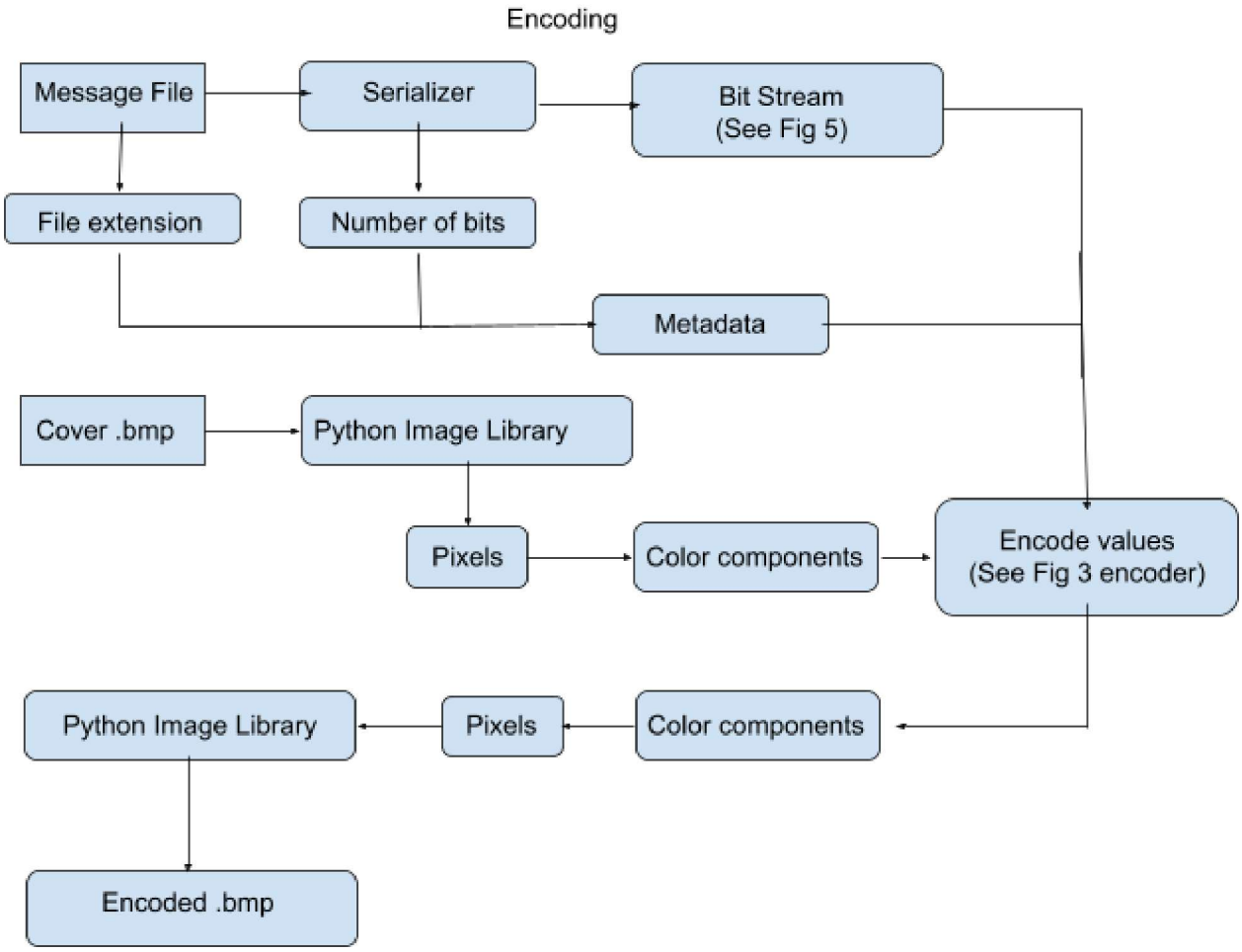
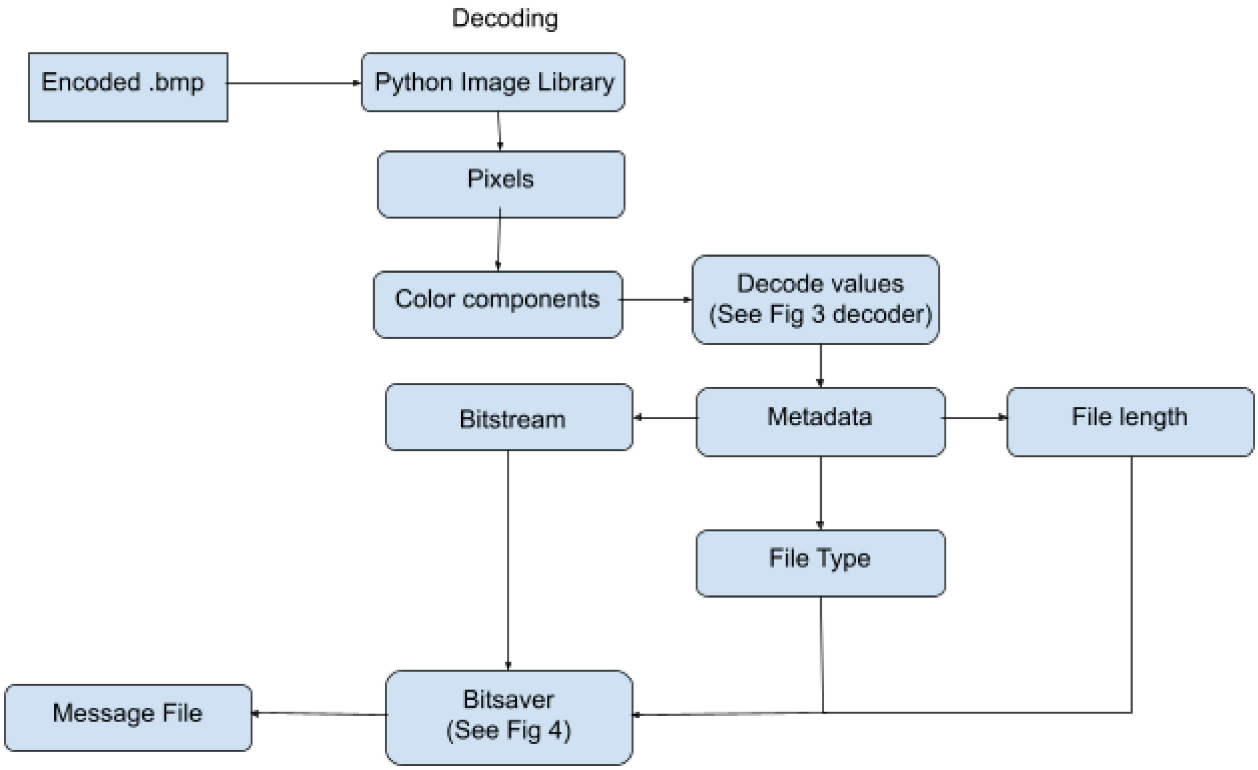


Fig 2. Decoding



Discussion

Bit depth method

For the bit depth method the argparse python library was utilized to take command line arguments as parameters. The parameters are a command (encode, decode), the cover (a full color .bmp file), the message (any type of file), and an optional argument to specify the bit depth which defaults to 8. If the command is to encode the file then python utilizes the os.getsize() method to determine the size of the message then compares that to the number of available bytes calculated by PIL.size()*⅜ to determine the number of bytes. A buffer of bytes for the metadata is accounted for as well. If the bit level is not specified the LSB of the color component is utilized (default 8) otherwise the following function snippets allow for encoding and decoding at any bit level of the color component.

Fig 3. Encoder/Decoder snippets

Cover is 1 byte color value, message is 1 bit from bitstream

```
def encoder(cover,message,level=8):
    cover = cover[:level-1]+message+cover[level:]
    return cover
#decodes 1 bit at specified bit level
def decoder(cover,level=8):
    return cover[level-1:level]
```

Fig 4. BitSaver snippets

```
def add_bits(self, bits, datatype=str, nlsb=None):
    if datatype == int:
        bits = '0'*7 + bin(bits)[2:] # add padding zero bits before
        bits = bits[-nlsb:]

    self.bits.append(bits)
    self.read_length += len(bits)

    if self.file_length is not None:
        if self.read_length >= self.file_length:
            self.bits = ''.join(self.bits)
            self.bits = self.bits[:self.file_length]
            self.finished = True
```

Fig 5. BitFeederString snippets

```
def _make_bitstring(self):
    bitsarray = []
    for byte in self.bytes:
        bits = bin(byte)[2:]
        bits = '0'*(8-len(bits)) + bits
        bitsarray.append(bits)
    self.bitstring = ''.join(bitsarray)
```

If the message will fit into the cover the metadata of the number of bits, and the file extension is encoded into the first 72 color component values. After the metadata is encoded the message file is serialized utilizing Rongrongs bit_operator library and encoded utilizing the above encoder function and a few utility functions to cycle ints to strings of binary bits. When the entire message is encoded into the cover. PIL.save() function generates a bitmap named “Encoded_output.bmp”. From this point multiple other messages can be encoded at various other unused bitlevels (Fig.2).

Fig6. Cover result encoding at various bit depths (8=LSB)

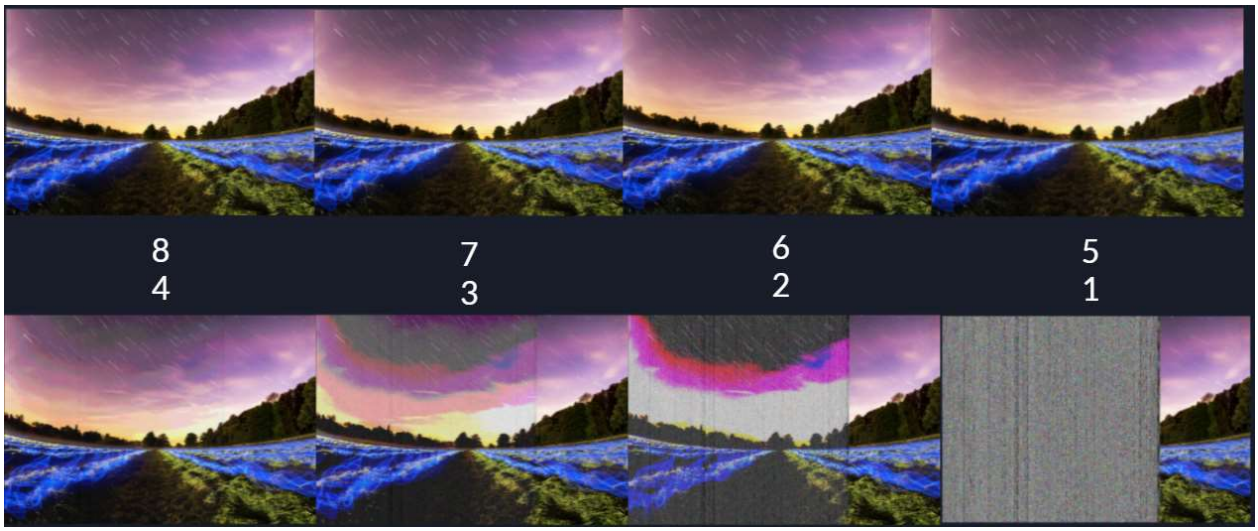


Fig7. Redundant copies of the same file LSB-bit depth 5

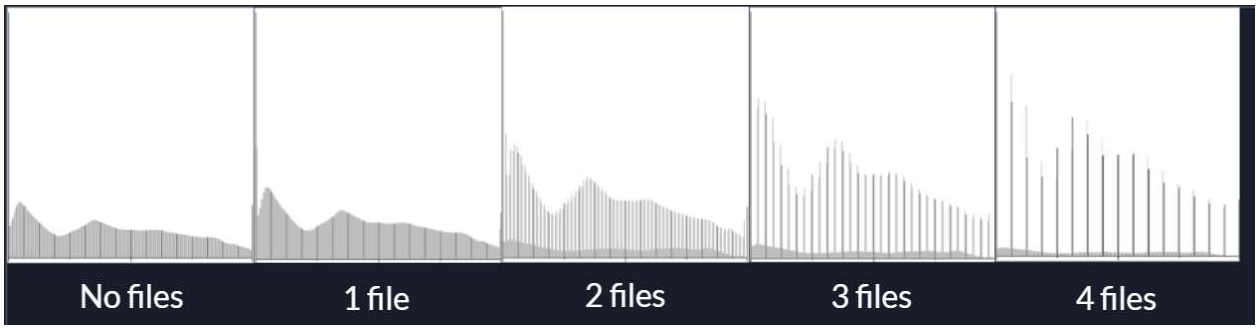
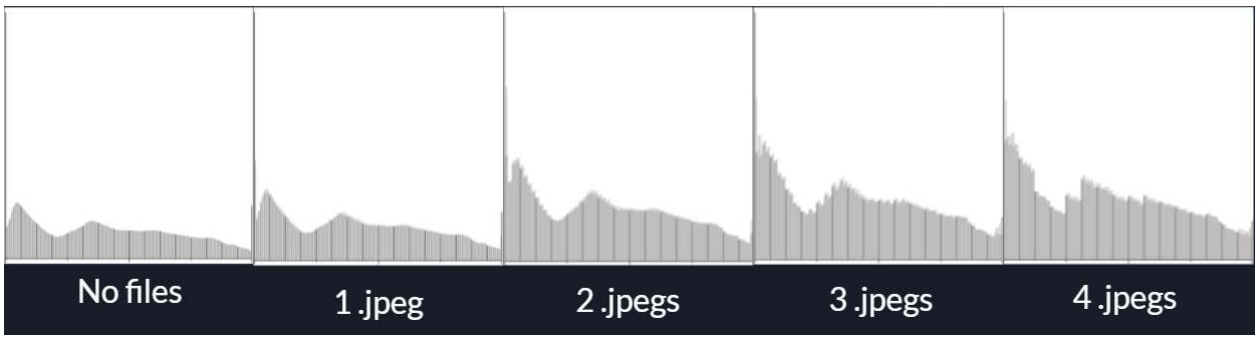


Fig8. Multiple different .jpeg files LSB-bit depth 5



If the command line argument is to decode the file the PIL.open() function is used on the encoded .bmp file and the metadata of the number of bits encoded, and file extension is decoded from the cover using get_length_from_file() and get_extension_from_file() functions at whatever bit depth specified. Once the metadata is in variables the index is set from the number of bits encoded and the above decode() function (Fig.1) is utilized to remove bits and placed into the bit_operators.BitSaver.add_bits() method. Once the number of encoded bits

matches the index the loop breaks and bit_operators.BitSaver.save() produces the output file with the correct extension.

Bit depth method enhancements vs LSB:

Capacity
- Multiple files in the same cover
- Theoretically 8x more capacity
- Practically 4x more capacity

Robustness
- Instead of LSB store in higher bit levels
- If a program sterilizes cover by wiping the LSB our data is still safe
- Multiple redundant copies of message (Obvious on histogram)

Security
- Serialized file (Rongrongs bit_operators.py) and metadata encoded into bits into our unique format

Statistical Hiding Method

In addition to the bit-depth method, we also implemented a program to store a hidden message using a statistical hiding approach. Compared to typical LSB hiding which hides message data in the LSB of each pixel component, statistical hiding combines the values of multiple pixel components and hides in the combined components' LSB. For example, if there are three pixel components to hide in: R=101, G=34, B=201, the average pixel-component value is 112. The LSB of the combined average is 0. If the message bit to hide in these values is 1, then the pixel components need to change so that the LSB of the average is 1. For example, a one could be added to each pixel component so that the new components are: R=102, G=35, B=202 and the average is 113 (LSB=1). Besides using an average, other ways of combining multiple pixels components can be used such as sum, min or max. In this project, we implemented both the average method and the sum method.

For our implementation of the average and sum method, there are two parameters. The number of LSB bits to hide in and the number of pixel components to use for the average or sum.

Mathematics of Statistical Hiding: Average Method

To implement the average method, we needed to figure out how to change the values of each pixel component to update the average with the message bit. To do this, we had to figure out what the difference was between the group of pixels and the desired average.

For statistical hiding, there are many possible numbers that have the right statistical value. For example, if the target message bit is a 0, there are many possible averages that end with a zero bit. To replace the pixels with the right average, we first had to figure out which average was closest to the average of the original pixel component values. We chose to find the average that was closest by first finding the value closest to the average that had an LSB of all zeros. For example, for 3 components, if the average of the original components was 132, the LSB was 3 and the hidden bits were '101', then the closest number with an LSB of all zeros was 128. After finding a number with all zeros, we add the message bits to the number to get the target average. In this example, the target average is 128+5 = 133. For n lsb, m pixel components with values x_i the target average is:

$$target\ average\ =\ \frac{\sum_{i=1}^{m} x_i}{m} - mod(\frac{\sum_{i=1}^{m} x_i}{m},\ 2^n)\ +\ message\ bits$$

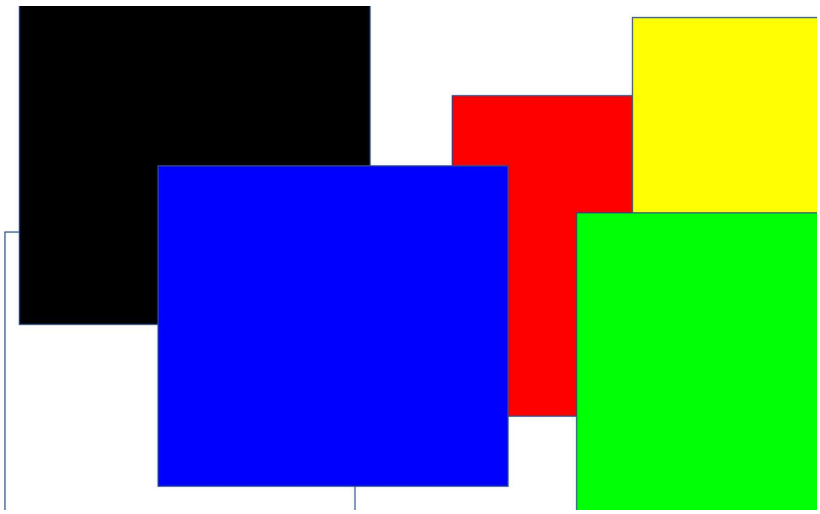
Once the target average is known, the difference of the current average from the target average needs to be calculated so that the original pixel components can be changed. The value of the difference for the sum of m pixel components with values x_i to create the target average is:

$$difference\ =\ target\ average\ \cdot m\ -\sum_{i=1}^{m} x$$

Once the difference is known between the target average and the sum of the pixel components, the original pixel components can be modified so that their average contains the message bits.

Implementation Challenges

The statistical hiding technique was correctly implemented but there was a confusing coding error that happened during implementation. The main issue was that the program was trying to subtract from pixel components with a value of zero and add to pixel components with a value of 255. This meant that the final pixel value was outside of the boundary of allowable pixel component values. For example, the program was trying to set values of the pixels to -1 and 256. When the python image library was trying to save these values, it was saving them as the closest allowable value (set -1 to 0 or 256 to 255). This meant that the average was not the correct message data. To fix this, conditions were set to prevent the code from trying to replace pixel components with a value of 0 and 255. To test the program's accuracy, a test message with large blocks of white and black pixels to make sure that the error had been fixed.



Test Color Image to Check Corrected Program (colortest.png)

Suggestions

For future work, we would like to try and implement a statistical hiding method that uses max and min values to hide the data. This might provide different perceptibility. For example, changes to the lower valued pixel components might be less noticeable than changes to higher level pixel components.

Another idea that we would like to implement in future work is pseudo-random pixel coordinate hiding. This might make hidden data less noticeable because there would not be a line where the hidden message ends in the cover file.

Examples

Statistical Hiding Method

This cat has the colortest.png image embedded using the averaging method with 3 LSB and 6 pixel components.



This cat has the colortest.png image embedded using the sum method with 3 LSB and 6 pixel components.



The sum method is significantly less perceptible than the average method at the same LSB and pixel-component number settings. Note that both the average and sum methods have the same capacity. The average method's perceptibility comes from how an average is calculated. To change an average of 5 pixels from 5 to 10, each pixel needs to change by 5. To change a sum from 5 to 10, each pixel needs to change by only 1.

Statistical Analysis Results

For bit depth hiding, the capacity for each message in bytes is:

$$capacity = w \cdot h \cdot 3 / 8$$

The capacity equation uses w and h for the width and height of the image. The 3 is for the 3 color channels and the 8 is to convert the capacity into bytes. Bit depth hiding uses 1 bit per pixel at the given depth.

For statistical hiding, the capacity for the average and sum method in bytes is:

$$capacity = w \cdot h \cdot 3 \cdot n / (8 \cdot m)$$

The statistical hiding capacity equation uses w and h for the width and height of the image. The 3 is for the 3 color channels and the 8 is to convert the capacity into bytes. The term n means the number of LSB bits to hide in the average and m is the number of pixel components used to create the average.

For the average method, the hiding becomes somewhat perceptible at 2 LSB. In this example, the pixelno setting is 100 and the distortion is visible as long horizontal lines.



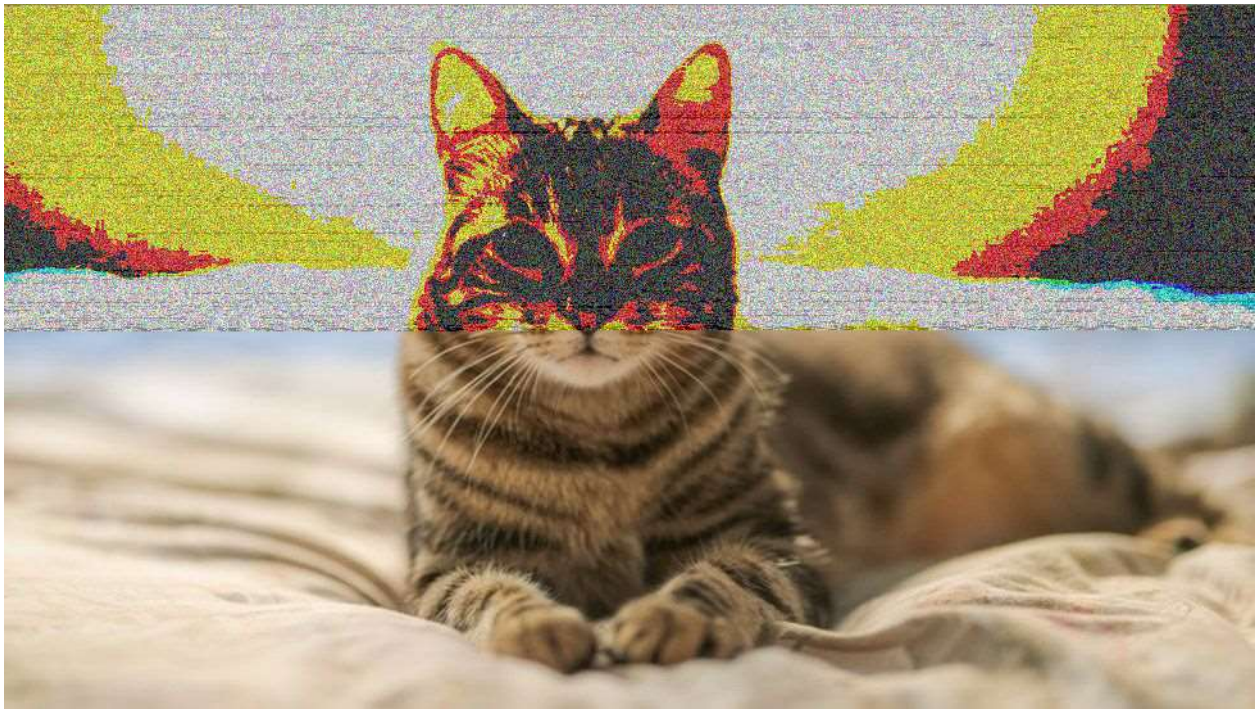
At 3 LSB, the distortion is more noticeable.



At 7 LSB, the distortion is very obvious.



Compared to regular LSB substitution (pixelno1) statistical hiding looks like it preserves the original colors better (LSB=7, pixelno1):



Team Member Contributions

Ben Baggs

- Prototyped image hiding technique
- Implemented Bit Depth Method
- Report Writing

Rongrong Wei

- Message serialization methods (convert message to bits, get n bits)
- Implemented Statistical Hiding Method
- Report Writing

Bibliography

N. Verma. 2011. Review of steganography techniques. In Proceedings of the International Conference & Workshop on Emerging Trends in Technology (ICWET '11). Association for Computing Machinery, New York, NY, USA, 990–993. DOI:https://doi-org.libweb.lib.utsa.edu/10.1145/1980022.1980237

Manocher C. Alipour, Bobby D. Gerardo, and Ruji P. Medina. 2020. LSB Substitution Image Steganography Based on Randomized Pixel Selection and One-Time Pad Encryption. In 2020 2nd International Conference on Big-data Service and Intelligent Computation (BDSIC 2020). Association for Computing Machinery, New York, NY, USA, 1–6. DOI:https://doi-org.libweb.lib.utsa.edu/10.1145/3440054.3440055

H. B. Kekre, A. A. Athawale, and U. A. Athawale. 2011. Increased cover capacity using advanced multiple LSB algorithms. In Proceedings of the International Conference & Workshop on Emerging Trends in Technology (ICWET '11). Association for Computing Machinery, New York, NY, USA, 25–31. DOI:https://doi-org.libweb.lib.utsa.edu/10.1145/1980022.1980029

Mark Rennel D. Molato, Bobby D. Gerardo, and Ruji P. Medina. 2018. Secured Data Hiding and Sharing using Improved LSB-based Image Steganography Technique. In Proceedings of the 4th International Conference on Industrial and Business Engineering (ICIBE' 18). Association for Computing Machinery, New York, NY, USA, 238–243. DOI:https://doi-org.libweb.lib.utsa.edu/10.1145/3288155.3288181